

trgn515_assign3_sub_github

ranzhu

2026-03-26

Task 1: Prepare a SLURM array job to perform the entire mapping pipeline

1. Quality control and filtering

```
#!/bin/bash
#SBATCH --account=msalomon_1816
#SBATCH --partition=main
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem=5G
#SBATCH --time=1:00:00
#SBATCH --job-name=fastqc
#SBATCH --output=/project2/msalomon_1816/ranzhu/assignment_3/logs/fastp%j.%a.out
#SBATCH --error=/project2/msalomon_1816/ranzhu/assignment_3/logs/fastp%j.%a.err
#SBATCH --array=1-8

source ~/.bashrc
conda activate /project2/msalomon_1816/trgn_515/conda_env/trgn515

# Put the input and output directories into variables for conciseness
in_dir=/project2/msalomon_1816/trgn_515/3_annotation/m_tuberculosis/fastq
out_dir=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/fastp

mkdir -p /project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/fastp

sample_list=/project2/msalomon_1816/ranzhu/assignment_3/sample_list.txt
sample=$(awk "NR==${SLURM_ARRAY_TASK_ID}" $sample_list)

fwd=$in_dir/${sample}_1.fastq.gz
rev=$in_dir/${sample}_2.fastq.gz

fastp -i $fwd -I $rev -o $out_dir/${sample}_1.fastq.gz -O $out_dir/${sample}_2.fastq.gz / -h $out_dir/
```

2. Mapping

```
#!/bin/bash
#SBATCH --account=msalomon_1816
#SBATCH --partition=main
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=5G
#SBATCH --time=1:00:00
#SBATCH --job-name=fastqc
#SBATCH --output=/project2/msalomon_1816/ranzhu/assignment_3/logs/alignment%j.%a.out
#SBATCH --error=/project2/msalomon_1816/ranzhu/assignment_3/logs/alignment%j.%a.err
#SBATCH --array=1-16

source ~/.bashrc
conda activate /project2/msalomon_1816/trgn_515/conda_env/trgn515

ref_genome=/project2/msalomon_1816/ranzhu/assignment_3/reference/GCF_000195955.2_ASM19595v2_genomic.fna
in_dir=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/fastp
out_dir=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/alignment

mkdir -p $out_dir

sample_list=/project2/msalomon_1816/ranzhu/assignment_3/sample_list.txt
sample=$(awk "NR==${SLURM_ARRAY_TASK_ID}" $sample_list)

fwd=$in_dir/${sample}_1.fastq.gz
rev=$in_dir/${sample}_2.fastq.gz

# Extract flowcell and lane information from fastq header
flowcell=$(zcat $fwd | head -n1 | cut -d':' -f3)
lane=$(zcat $rev | head -n1 | cut -d':' -f4)

# Define Read Group variables
rg_id=${flowcell}.${lane}
pu=${flowcell}.${lane}.${sample}
pl=ILLUMINA

bwa mem -t 4 \
  -R "@RG\tID:$rg_id\tPU:$pu\tPL:$pl\tSM:${sample}" \
  $ref_genome \
  $in_dir/${sample}_1.fastq.gz \
  $in_dir/${sample}_2.fastq.gz \
  | samtools view -bS - \
  | samtools sort \
  -@ 4 \
  -T $out_dir \
  -O bam \
  -o $out_dir/${sample}.bam
```

3. Marking duplicates

```
#!/bin/bash
#SBATCH --account=msalomon_1816
#SBATCH --partition=main
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=5G
#SBATCH --time=1:00:00
#SBATCH --job-name=fastqc
#SBATCH --output=/project2/msalomon_1816/ranzhu/assignment_3/logs/dedup%j.%a.out
#SBATCH --error=/project2/msalomon_1816/ranzhu/assignment_3/logs/dedup%j.%a.err
#SBATCH --array=1-8

source ~/.bashrc
conda activate /project2/msalomon_1816/trgn_515/conda_env/trgn515

# paths
IN=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/alignment
OUT=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/alignment

sample_list=/project2/msalomon_1816/ranzhu/assignment_3/sample_list.txt
sample=$(awk "NR==${SLURM_ARRAY_TASK_ID}" $sample_list)

# Mark duplicates using samtools
samtools collate -Ou $IN/$sample.bam | \
  samtools fixmate -m - -u | \
  samtools sort - -u | \
  samtools markdup - $OUT/$sample.dedup.bam

samtools index $OUT/$sample.dedup.bam
```

4. Indel re-alignment

```
#!/bin/bash
#SBATCH --account=msalomon_1816
#SBATCH --partition=main
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=5G
#SBATCH --time=1:00:00
#SBATCH --job-name=indel
#SBATCH --output=/project2/msalomon_1816/ranzhu/assignment_3/logs/indel%j.%a.out
#SBATCH --error=/project2/msalomon_1816/ranzhu/assignment_3/logs/indel%j.%a.err
#SBATCH --array=1-8

set -euo pipefail

source ~/.bashrc
```

```

conda activate /project2/msalomon_1816/trgn_515/conda_env/trgn515

IN=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/alignment
OUT=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/alignment
ref_genome=/project2/msalomon_1816/ranzhu/assignment_3/reference/GCF_000195955.2_ASM19595v2_genomic.fna

sample_list=/project2/msalomon_1816/ranzhu/assignment_3/sample_list.txt
sample=$(awk "NR==${SLURM_ARRAY_TASK_ID}" $sample_list)

abra2 \
  --in "$IN/$sample.dedup.bam" \
  --out "$OUT/$sample.indel.bam" \
  --ref "$ref_genome" \
  --threads 4

samtools index "$OUT/$sample.indel.bam"

```

5 & 6. Variant calling & soft-filtering

```

#!/bin/bash
#SBATCH --account=msalomon_1816
#SBATCH --partition=main
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=4
#SBATCH --mem=5G
#SBATCH --time=1:00:00
#SBATCH --job-name=variant
#SBATCH --output=/project2/msalomon_1816/ranzhu/assignment_3/logs/variant%j.%a.out
#SBATCH --error=/project2/msalomon_1816/ranzhu/assignment_3/logs/variant%j.%a.err
#SBATCH --array=1-8

source ~/.bashrc
conda activate /project2/msalomon_1816/trgn_515/conda_env/trgn515

ref_genome=/project2/msalomon_1816/ranzhu/assignment_3/reference/GCF_000195955.2_ASM19595v2_genomic.fna
input_dir=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/alignment
out_dir=/project2/msalomon_1816/ranzhu/assignment_3/m_tuberculosis/vcf

mkdir -p $out_dir

sample_list=/project2/msalomon_1816/ranzhu/assignment_3/sample_list.txt
sample=$(awk "NR==${SLURM_ARRAY_TASK_ID}" $sample_list)

input_bam=$input_dir/$sample.indel.bam

bcftools mpileup \
  -f $ref_genome \
  -a AD,INFO/AD,ADF,INFO/ADF,ADR,INFO/ADR,DP \
  -q 0 \

```

```

-Q 20 \
-Ou \
$input_bam \
| bcftools call \
-m \
-v \
-V indels \
-Ou \
| bcftools norm \
-f $ref_genome \
-Oz -o $out_dir/$sample.vcf.gz
tabix -p vcf $out_dir/$sample.vcf.gz

bcftools filter -m+ -s 'MinMQ' -e 'INFO/MQ < 20' \
$out_dir/$sample.vcf.gz \
| bcftools filter -m+ -s 'QUAL' -e 'QUAL < 20' \
| bcftools filter -m+ -s 'MinDP' -e 'INFO/DP < 30' \
| bcftools filter -m+ -s 'minAD' -e 'FMT/AD[:1] < 5' \
| bcftools filter -m+ -s 'minADF' -e 'FMT/ADF[:1] < 3' \
| bcftools filter -m+ -s 'minADR' -e 'FMT/ADR[:1] < 3' \
-Oz -o $out_dir/$sample.filtered.vcf.gz

tabix -p vcf $out_dir/$sample.filtered.vcf.gz

```

7. Index the vcf file (tabix -p vcf <input_file.vcf.gz>)

```

tabix -p vcf $out_dir/$sample.vcf.gz
tabix -p vcf $out_dir/$sample.filtered.vcf.gz

```

Task 2: Merge the VCF files from the T85 strain and the CDC strain

1. Paste your variant merging command (5pts)

merge CDC

```

bcftools merge -f .,PASS -Oz -o CDC.merged.vcf.gz \
  CDC06E.filtered.vcf.gz \
  CDC07E.filtered.vcf.gz \
  CDC1551E.filtered.vcf.gz \
  CDC47E.filtered.vcf.gz

tabix -p vcf CDC.merged.vcf.gz

```

merge T85

```
bcftools merge -f .,PASS -Oz -o T85.merged.vcf.gz \  
T8501E.filtered.vcf.gz \  
T8505E.filtered.vcf.gz \  
T8538E.filtered.vcf.gz \  
T85E.filtered.vcf.gz  
  
tabix -p vcf T85.merged.vcf.gz
```

2. Describe each column of the resulting VCF file. How does it differ from a single-sample VCF file? (5pts)

- The multi-sample VCF has the same first 9 columns as a single-sample VCF: CHROM, POS, ID, REF, ALT, QUAL, FILTER, INFO, FORMAT. The difference is that instead of one SAMPLE column at the end, there are now multiple sample columns (one per sample, e.g. CDC06E, CDC07E, CDC1551E, CDC47E).
- Each sample column contains that sample's individual genotype information following the FORMAT field. The INFO column now reflects aggregated statistics across all samples combined, rather than a single sample.

3. Should we use the INFO column or the FORMAT column to filter specific samples? Why? (5pts)

- We should use the FORMAT column to filter specific samples.
- The INFO column contains site-level statistics aggregated across all samples (e.g. overall allele frequency, total depth), so it doesn't distinguish between individual samples. The FORMAT column defines per-sample fields (e.g. GT, DP, GQ), allowing us to filter based on each sample's own genotype and quality information independently.

Task 3: Removing variants that fall within hypervariable regions.

1. Hard-filter the variants from your multi-sample VCF that fall within the hypervariable regions in the .bed file. Attach your command (5pts)

```
bcftools view -T ~/project2/msalomon_1816/trgn_515/3_annotation/m_tuberculosis/reference/mtb.h37rv.hyper  
CDC.merged.vcf.gz -Oz -o CDC.merged.hardfiltered.vcf.gz  
  
bcftools view -T ~/project2/msalomon_1816/trgn_515/3_annotation/m_tuberculosis/reference/mtb.h37rv.hyper  
T85.merged.vcf.gz -Oz -o T85.merged.hardfiltered.vcf.gz
```

Task 4: Variant annotation file

Find in NCBI and download the annotation file in GFF format for ASM19595v2, and fill in variable `gff_file` (5pts) `gff_file=/path/to/gff_file`

```
gff_file=/project2/msalomon_1816/trgn_515/3_annotation/m_tuberculosis/reference/Mycobacterium_tuberculosis
```

Task 5: Variant functional annotation

Use one of VEP, bcftools, or SnpEff to annotate the variants from the multi-sample filtered VCF file (5pts)

```
zcat CDC.merged.hardfiltered.vcf.gz | \
sed 's/NC_000962.3/Chromosome/g' | \
snpEff eff Mycobacterium_tuberculosis_h37rv > CDC.annotated.vcf

zcat T85.merged.hardfiltered.vcf.gz | \
sed 's/NC_000962.3/Chromosome/g' | \
snpEff eff Mycobacterium_tuberculosis_h37rv > T85.annotated.vcf
```

Task 6: Parse required information from VCF files

Use any of the options listed above to parse the variant functional annotation from the VCF file. Attach your parsing command to the assignment (5pts)

```
bcftools query -f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE=%GT]\t%INFO/ANN\n' CDC.annotated.vcf | \
grep -v "0/1" | \
awk -F'\t' '{
    split($9, anns, ",");
    split(anns[1], ann, "|");
    print $1"\t"$2"\t"$3"\t"$4"\t"$5"\t"$6"\t"$7"\t"$8"\t"ann[2]"\t"ann[4]
}' > CDC_results.txt

bcftools query -f '%CHROM\t%POS\t%REF\t%ALT[\t%SAMPLE=%GT]\t%INFO/ANN\n' T85.annotated.vcf | \
grep -v "0/1" | \
awk -F'\t' '{
    split($9, anns, ",");
    split(anns[1], ann, "|");
    print $1"\t"$2"\t"$3"\t"$4"\t"$5"\t"$6"\t"$7"\t"$8"\t"ann[2]"\t"ann[4]
}' > T85_results.txt
```

Task 7: Detect variants likely to be associated to rifampicin resistance.

1. What criteria does a variant need to meet to be a good candidate for rifampicin resistance in our experiment? (10pts)

- For a variant to be a good candidate for rifampicin resistance, it should be present in the resistant strains but absent in the susceptible controls (CDC1551E / T85E). If the control strains also carry the variant, it's likely just a background mutation unrelated to resistance.
- It should also be a non-synonymous variant, such as a missense mutation. Synonymous variants don't change the protein sequence because the amino acid stays the same despite the DNA change, so they're unlikely to have any effect on drug resistance.
- Thirdly, it should be in a gene that rifampicin actually targets. Rifampicin works by blocking RNA polymerase, so mutations in *rpoB* are the most likely candidates since that's the gene that encodes the RNA polymerase beta subunit.

2. What are the genes with the highest number of variants? (5pts)

- CDC lineage: Rv2082 and PPE9 have the highest number of variants (8 each), followed by *ppsA* and *pks12* (5 each).
- T85 lineage: *lppB* has the highest number of variants (19), followed by PPE55 (17) and Rv0275c (10).

3. What are the genes with the highest number of non-synonymous variants? In how many samples are these mutations present? (5pts)

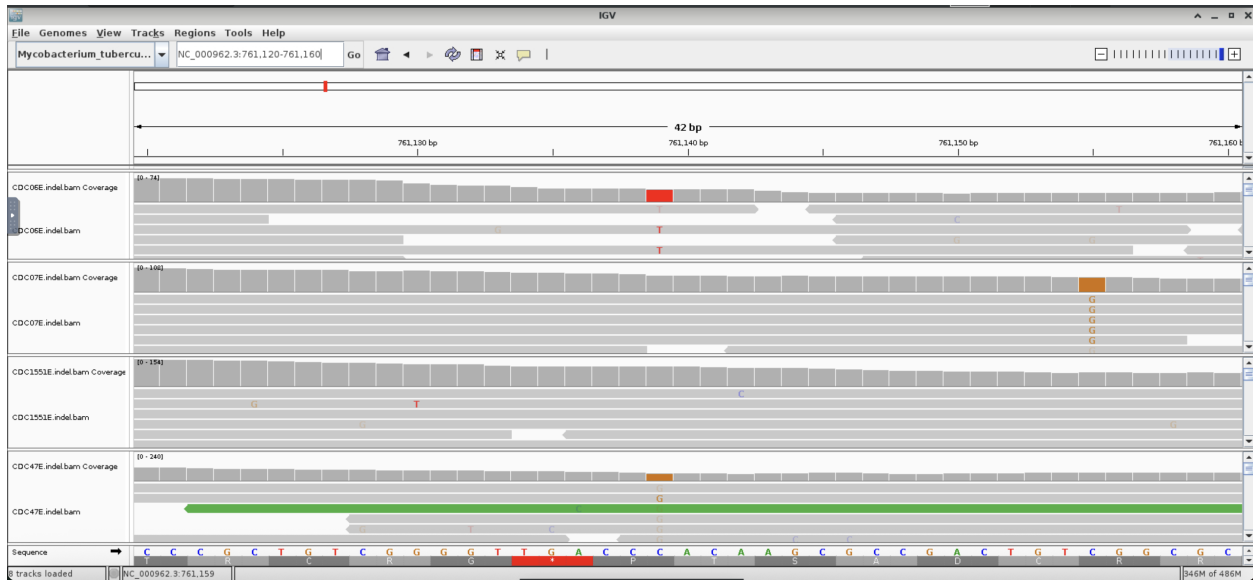
- For CDC lineage, PPE9 has the highest number of non-synonymous variants (7 missense), present in all 4 samples including the control. For T85 lineage, *lppB* has the highest number (12 missense), also present across all 4 samples including the control. However, since both genes carry mutations in the susceptible controls as well, they are likely background mutations rather than resistance-associated variants.

4. What are the genes with the highest proportion of non-synonymous variants to synonymous variants? (5pts)

- For CDC lineage, PPE9 has the highest proportion of non-synonymous to synonymous variants with a ratio of 7:1 (7 missense : 1 synonymous), followed by several genes with a ratio of 2:1 including *ponA1*, *mmpL2*, and *glpK*. Rv2082 has a ratio of 5:3.
- For T85 lineage, PPE9 also has the highest proportion with a ratio of 6:1, followed by Rv2082 at 4:1 and *espK* at 3:1.

5. Review the variants within those genes in IGV and determine if the alignment is reliable (5pts)

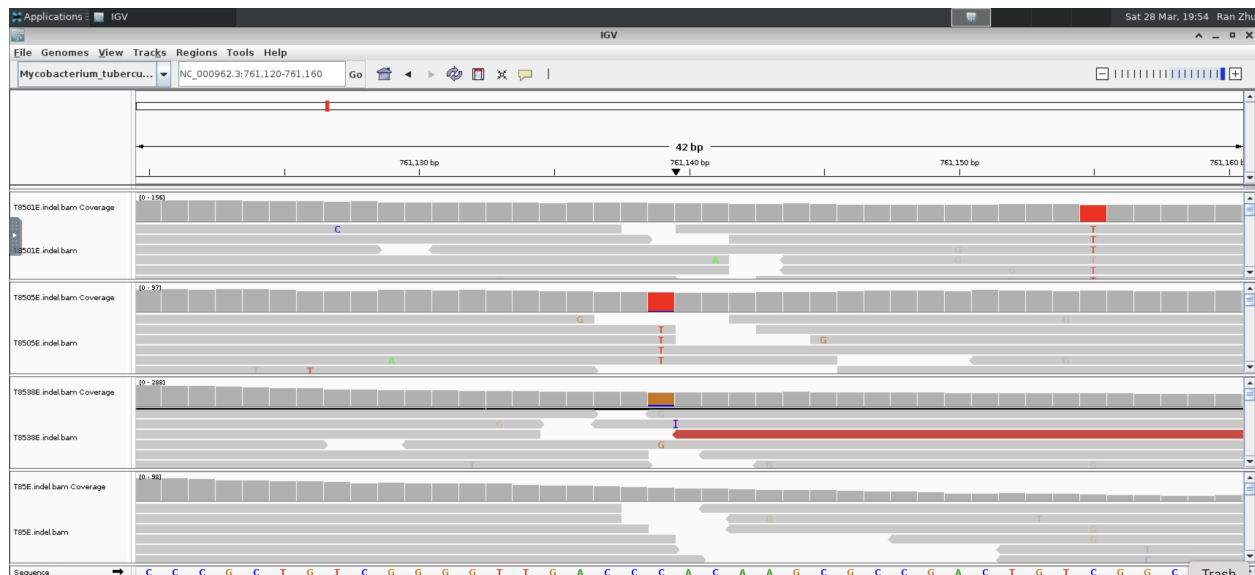
- CDC lineage: NC_000962.3:761,000-761,200



- CDC lineage: NC_000962.3:761,120-761,160



- For CDC lineage, in the rpoB region, at position 761,001, a variant is present only in CDC07E but not in the control CDC1551E. At position 761,139, CDC06E shows a consistent C→T substitution and CDC47E shows a consistent C→G substitution across all reads, while CDC1551E only shows a single read with a T substitution. Since this change is only supported by one read out of many, it is likely sequencing noise rather than a true variant.
- T85 lineage: NC_000962.3:761,120-761,160



- For T85 lineage, at position 761,139, T8505E shows a consistent C→T substitution and T8538E shows a consistent C→G substitution across all reads, while the control T85E shows a clean reference sequence with no variant at this position.
- Therefore, the alignment appears reliable as the variants are supported by multiple reads and coverage is adequate across all samples.

Task 8: Find pathogenic variants.

1. Write your bcftools command to select variants. Return the chromosome and position of the resulting variants. (10pts)

```
bcftools view \
-i '(AF < 0.01 || AF = ".") && CLNSIG ~ "Pathogenic"' \
HCC1395_combined.down.gnomad.clinvar.vcf.gz | \
bcftools query -f '%CHROM\t%POS\n'
```

- chr17 7675088
- chr17 43057078

2. Search for the exact genomic coordinates of these variants on the ClinVar website and answer the following questions:

1. What genes do the variants fall in? (5pts)

- The two variants fall in TP53 (chr17:7675088) and BRCA1 (chr17:43057078).

2. Find information about these genes in the ClinVar DB and briefly describe the function of the gene (5pts)

- TP53 encodes the tumor suppressor protein p53, which helps regulate the cell cycle, DNA repair, and apoptosis. When DNA damage occurs, p53 acts as a transcription factor to either halt cell division or

trigger cell death, preventing mutations from accumulating and leading to cancer. Mutations in TP53 are linked to many cancers, including Li-Fraumeni syndrome.

- BRCA1 encodes a nuclear protein that helps maintain genomic stability and acts as a tumor suppressor. It's part of a larger protein complex called BASC and is involved in DNA double-strand break repair and recombination. Mutations in BRCA1 account for around 40% of inherited breast cancers and over 80% of inherited breast and ovarian cancers.

3. What is the specific molecular consequence of each mutation (e.g., missense, nonsense/stop-gain, frameshift)? (5pts)

- TP53 (chr17:7675088): The variant c.524G>A (p.Arg175His) is a missense variant, where a single nucleotide change results in an arginine to histidine substitution at amino acid position 175 in the DNA-binding domain.
- BRCA1 (chr17:43057078): The variant c.5251C>T (p.Arg1751Ter) is a nonsense variant (stop-gain), where a single nucleotide change introduces a premature stop codon, resulting in a truncated, non-functional protein.

4. What is the evidence of these variants being pathogenic?

- For TP53, the variant has been reported in multiple families with Li-Fraumeni syndrome and segregates with disease. In vitro studies show loss of tumor suppressor activity. It is located at a known mutational hotspot and is extremely rare in the general population (gnomAD AF = 0.00001).
- For BRCA1, the variant has been reported in numerous individuals with breast and/or ovarian cancer. Functional studies show the variant results in a truncated non-functional protein. It is absent from large population databases and has been classified as Pathogenic by multiple independent laboratories.

5. Why did we filter out variants with a gnomAD AF > 1%? (10pts)

- Variants with a gnomAD AF > 1% are common in the general healthy population, meaning they are unlikely to be rare pathogenic driver mutations. By filtering them out, we can focus on rare variants that are much more likely to be disease-causing somatic or germline mutations responsible for cancer.

Task 9: Inferring pathogenic variants

1. Find information about those 5 genes in ClinVar. What genes are more likely oncogenic based on the ClinVar annotations? (10pts)

- Top 5 genes are: TBX2, VWA8, MYADML2, URAD, MYH4
- For TBX2, a missense variant (G>C) in exon 1, associated with vertebral anomalies and T-cell dysfunction.
- For VWA8, a splice site variant that modifies a natural donor site, associated with retinitis pigmentosa and cleft lip/palate.
- For MYADML2, a missense variants only, all of uncertain significance with no specific disease association.
- For URAD, a missense variant (C>T) of uncertain significance with no specific disease association.
- For MYH4, a missense variants only, all of uncertain significance or benign with no specific disease association.

- Therefore, based on ClinVar annotations, none of the 5 genes have clearly annotated cancer-associated Pathogenic variants. TBX2 is the most likely oncogenic candidate, as it is located in the 17q23 region which is commonly amplified in breast cancer, but no specific cancer Pathogenic variants are recorded in ClinVar for this gene.

2. From those likely oncogenic genes, what mutations were not annotated in ClinVar in our previous analysis? (5pts)

- The TBX2 variant at chr17 position 61401752 was not annotated in our previous ClinVar analysis in Task 8, because Task 8 only returned variants with explicit Pathogenic classifications in ClinVar (TP53 and BRCA1).

3. What is the functional consequence of these mutations? (5pts)

- The TBX2 variant (chr17 position 61401752, T>C) is a missense variant predicted to be deleterious by SIFT and probably damaging by PolyPhen, with a CADD score of 33, indicating it is likely to disrupt protein function.